

TEST PLAN

Language Learning Hub- Flashcard CRUD, AI Generation, Sharing & Export

Testing Portfolio Project

Github: [LLH_Mannual_API_testing_project](#)

Github Web Project: [Language-learning-hub-for-quizz](#)

Url: [link web](#)

Requirements: Requirement Analysis, API-CONTRACT, API-CONVENTIONS.

1. Test Objectives.....	3
2. Scope.....	3
2.1 In-Scope API.....	3
2.2 Functional areas' scope.....	3
3. Coverage / Test Conditions.....	4
4. Test Approach.....	5
4.1 Functional contract testing.....	5
4.2 Boundary and data-driven strategy.....	6
4.3 Workflow testing.....	6
4.4 Authorization testing.....	6
5. Environment and Actor Model.....	7
5.1 Environment.....	7
5.2 Actors.....	7
6. Fixture.....	7
7. High-Level Postman Topology.....	7
8. Dependencies and Preconditions.....	8
9. Cleanup Strategy.....	8
10. Entry, Exit.....	8
10.1 Entry criteria.....	8
10.2 Exit criteria.....	9

1. Test Objectives

- Verify that an Active Learner can manage Flashcards within Decks they own.
- Verify that validation, normalization, and duplicate protection preserve data integrity.
- Verify that AI Preview only generates transient content without persistence.
- Verify that AI Save stores selected content accurately and guarantees all-or-nothing atomicity.
- Verify that the owner controls the sharing lifecycle and token revocation.
- Verify that public responses contain only read-only public projections.
- Verify that export produces valid XLSX files and does not alter data.
- Verify authentication, roles, account status, and object ownership at the API layer.
- Verify that responses do not leak credentials, tokens, raw SRS data, or internal details.

2. Scope

2.1 In-Scope API

API ID	Method	Path
API-DOC-010	POST	/decks/{deckId}/flashcards
API-DOC-011	GET	/decks/{deckId}/flashcards
API-DOC-012	PATCH	/flashcards/{flashcardId}
API-DOC-013	DELETE	/flashcards/{flashcardId}
API-DOC-014	POST	/ai/flashcard-previews
API-DOC-015	POST	/generated-flashcards
API-DOC-016	POST	/decks/{deckId}/share
API-DOC-017	DELETE	/decks/{deckId}/share
API-DOC-018	GET	/shared-decks/{shareToken}
API-DOC-019	GET	/decks/{deckId}/export

2.2 Functional areas' scope

In scope

- Manual Flashcard list/create/edit/delete operations and empty states.

- Required/trim/length validation, Unicode/multiline preservation, and normalized duplicates.
- AI vocabulary validation, preview generation, failed-item handling, and zero persistence.
- Saving to an owned existing Deck or a new Deck in an owned Folder.
- Atomicity and zero-write verification for rejected mutations.
- Sharing enable/idempotency/disable/re-enable, token revocation, and rotation.
- Anonymous public read with an allow-list projection.
- XLSX response, filename/headers, and workbook content.
- Missing/invalid authentication, role restrictions, and deactivated accounts.
- Cross-owner access across high-risk read/write/share/export operations.
- Response envelope, correlation ID, and sensitive-data minimization.

Out of scope

- Folder/Deck management beyond what is required for fixture preparation and cleanup.
- Full learning/review/SRS workflows, Dashboard, Notifications, and Account Management.
- UI rendering and browser-only behaviors.
- Load, stress, rate-limiting, performance SLAs, and concurrency testing.
- Fault injection for the AI provider, database, or export services.
- PATCH mass-assignment testing.
- Malformed/unknown share-token validation outside of active/revoked lifecycle.
- Full cross-module end-to-end regression and production execution.

3. Coverage / Test Conditions

Coverage ID	Test condition	Product basis	Expected outcome to verify
FRC-C OV-01	Owned Flashcard lifecycle	BR-02.1, FR-DOC-009..012	Owner can list/create/edit/delete; empty state and deletion omission behave correctly
FRC-C OV-02	Validation and duplicate integrity	BR-02.2, BR-02.4, field constraints	Required/trim/limit/normalization are enforced; rejection does not alter baseline
FRC-C OV-03	Learning-state integrity	BR-02.5, FR-DOC-010..012	Create initializes state; edit preserves state; delete

FRC-C OV-04	AI input and preview safety	BR-02.5A, BR-02.5B, FR-DOC-013..014	Input is parsed/normalized/bounded; preview/failedItems are correct with zero writes
FRC-C OV-05	AI save target and atomicity	BR-02.5C, FR-DOC-015	Only one target mode is valid; selected content is completely saved or rolled back
FRC-C OV-06	Owner sharing lifecycle	BR-02.6, BR-02.7, FR-DOC-016, FR-DOC-018	Enable/idempotency/disable/re-e nable and token lifecycle work correctly
FRC-C OV-07	Public access and privacy	BR-02.6, FR-DOC-017	Active link is anonymous/read-only; revoked link is unavailable; public allow-list enforced
FRC-C OV-08	Non-mutating XLSX export	BR-02.8, FR-DOC-019	Eligible Deck generates valid workbook; rejected export does not alter data
FRC-C OV-09	Authenticatio n, role, account status	Permission matrix/API conventions	Missing/invalid session, Admin, and deactivated Learner are blocked per contract
FRC-C OV-10	Cross-owner authorization	BR-02.1, permission matrix	Actor cannot read/mutate another Learner's resource; no object disclosure beyond contract
FRC-C OV-11	Response and data minimization	API conventions/public /export rules	Envelope/projection is correct; no secrets, raw SRS, or internal details exposed

4. Test Approach

4.1 Functional contract testing

- method/route behavior;
- authentication and authorization outcomes;
- approved success/error contracts;
- response envelopes and field projections;
- persistence or non-mutating side effects;
- downstream effects when required by specifications.

4.2 Boundary and data-driven strategy

Dimension	Coverage strategy
Required field	Omitted, null, empty, whitespace-only, and valid
String length	Representative valid, approved maximum, and maximum + 1
Normalization	Leading/trailing spaces, collapsed spaces, case/Unicode handling per contract
Collection limit	Empty, valid, exact maximum, and maximum + 1
Target selection	One valid target; missing or mutually conflicting modes
Duplicate	Unique, normalized duplicate, and duplicate within atomic batch
Language data	Japanese, Chinese, multiline, and language-inconsistent input
State	Success-state for accepted input; zero-write/baseline-unchanged for rejections

4.3 Workflow testing

- create/edit/delete followed by retrieval;
- preview followed by explicit save;
- enable → public read → disable → revoked read → re-enable;
- denied mutation followed by owner-side verification;
- export followed by verifying application state remains unchanged.

4.4 Authorization testing

Apply an actor/operation/resource model:

- valid, missing, or invalid sessions;
- roles and account statuses;
- resources belonging to owner, another Learner, or non-existent;
- read versus mutation operations;
- object-existence disclosure;
- owner-side non-mutation verification following denied writes.

5. Environment and Actor Model

5.1 Environment

Component	Requirement
Backend	Build under test running in local/non-production environment
AI provider	Valid configuration verified prior to positive AI coverage execution
Postman	Environment targeting the correct endpoint; secrets stored strictly in local/current secret storage

5.2 Actors

Actor	Role/status	Coverage purpose
Learner A	Learner / Active	Owner for functional success and state verification
Learner B	Learner / Active	Independent owner for cross-owner coverage
Learner C	Learner / Deactivated	Account restrictions
Admin	Admin / Active	Learner-only role restrictions
Anonymous	No session	Public shared access

6. Fixture

Minimum preparation:

- owner hierarchy containing a Deck with baseline Flashcards;
- owned empty Deck for empty-state and export eligibility testing;
- owned disposable resources for destructive/stateful coverage;
- foreign hierarchy belonging to another Learner for object authorization testing;

7. High-Level Postman Topology

A single physical collection is used. High-level groupings:

DOC API Collection

- Setup and Cleanup
- Manual Flashcard Lifecycle
- AI Preview & Save
- Sharing Lifecycle
- Public Shared Access
- Export
- Authorization & Object Ownership

8. Dependencies and Preconditions

- Product requirements, API contracts, and ambiguity resolutions affecting expected behavior have been approved.
- Backend build, database schema, and target environment are compatible.
- Actor accounts and Active/Deactivated/Admin statuses are prepared.
- AI provider is operational for successful generation coverage.
- Baseline fixtures have known ownership and state.
- Read-only verification is available for state/count/atomicity when APIs provide insufficient observability.
- Tools and safe locations are available to save and inspect XLSX files.
- Approved test cases and test data are ready prior to Test Execution.
- No real credentials exist in shared artifacts.

9. Cleanup Strategy

- Clean up only resources created by or dedicated to the current execution run.
- Prioritize deleting children before parents unless an approved cascading contract has been verified.
- Disable active shares before deleting resources if required by lifecycle rules.

10. Entry, Exit

10.1 Entry criteria

- Test Plan and in-scope coverage have been approved.
- Product requirements and API contracts are sufficiently clear to determine expected behavior.
- Target build and environment are ready.
- Actor, fixture, and test-data strategies are prepared.
- Approved test cases and test data are available prior to execution.
- Read-only database verification and XLSX storage capability are confirmed where necessary.

- Secret handling and cleanup safeguards have been reviewed.

10.2 Exit criteria

- All mandatory in-scope coverage conditions have been executed via approved test cases.
- Functional, authorization, atomicity, privacy, and non-mutation assertions have been evaluated.
- No blocker/critical defects remain without an explicit risk-acceptance decision.
- Blocked or unexecuted mandatory coverage items have documented reasons and designated owners.
- TBD-EXPORT-NAME-01, if still open, is reported as a scoped blocked assertion and does not block unrelated Export or API coverage.
- Test-created data has been cleaned up or documented.
- Test results, evidence, and defects are finalized and traced to test cases.